NETVERSE LAB / DOCUMENTATION

DEVELOPER MANUAL

Architecture • Testing • Release engineering

A practical reference for the local-first networking learning and simulation platform.

Extended educational release • October 27, 2026

Illustrated technical documentation • Educational prototype

Edition: Extended educational release (Phases 1–14 candidate)
Source baseline: Phase 1–8 commit `e77c5fd`; accepted hotfix commit `1212d6c`; extended features currently pending final commit
Owner-reported Ubuntu verification: Node `v24.21.0`; npm `11.19.0`; Vite `8.3.5`; Vitest `5.0.5`; `npm run lint` 0 warnings/errors; `npm run build` successful; `npm test` **62 passing**
Documentation date: 27 October 2026

> **Scope statement:** This manual documents the *implemented source*, not the intended complete networking curriculum. NETVERSE currently has a working simulated IPv4/ARP/ICMP core with independent teaching experiments. Browser-simulator networking and real Linux namespaces must remain separate trust boundaries.

Contents

1. Project mission and verified state
2. Clone, installation and scripts
3. Source architecture and module ownership
4. Data schema and persistence
5. Core packet processing
6. UI and state management
7. Learning, assistance and case assessment
8. Phases 9–14 feature boundaries
9. Quality assurance and acceptance
10. Debugging and failure investigation
11. Release engineering, GitHub and final commit
12. Security, performance and accessibility
13. Known issues and extension roadmap
14. Rebuild checklist

---

1. Project mission and verified state

NETVERSE LAB is a **local-first React + TypeScript virtual networking classroom**. Learners work through theory, construct topologies, modify interface configurations, execute a strictly allow-listed *virtual* terminal, inspect events and repair predefined faults.

Delivered modules:

Phase	Implemented scope	Fidelity status
0	Vite/React/TypeScript toolchain, Oxlint, Vitest, Git	Operational

Phase	Implemented scope	Fidelity status
1	Sidebar/dashboard/build-oriented UI	Operational
2	Three introductory nine-step lessons, quiz/progress	Partial curriculum
3	SVG topology; host, server, router, switch; editable ports/cables	Small-scale 2D
4	State-based IPv4, ARP/ICMP, deterministic event trace	Simplified protocol model
5	Address/prefix/interface/default-gateway configuration	Connected/default routes
6	Allowed virtual terminal commands	Not a system shell
7	Three graded fault fixtures	Evidence-length, not semantic grading
8	Packet trace and playback	Simplified switching/traceroute
9	Max-min bandwidth-sharing lab	Single-bottleneck educational model
10	Infrastructure/equipment explorer	Schematics and analogies, not 3D optics
11	Firewall decision lab	Separate from IPv4 packet engine
12	VLAN, TCP handshake, IPv6 prefix labs	Independent concept experiments
13	Optional isolated Linux namespaces script	Real kernel only upon explicit run
14	Verification scripts and acceptance checklist	No production/security audit

The owner's screenshot review confirms the Extended UI renders in Firefox, including Guided/Independent, Patch, Build, Packet Inspector, Performance, Infrastructure and Security. The owner reported all tests passed. **No remote GitHub push or main-branch merge has been evidenced as of this document.**

2. Clone, installation and scripts

From the published GitHub repository

After the repository has been created and the final branch has been pushed, copy its **actual HTTPS clone URL** from GitHub. The following URL is an example if the maintainer publishes the repository under that name:

```
cd ~/Documents
git clone https://github.com/derrenderrick00-ux/netverse-lab.git
cd netverse-lab
node --version
npm --version
npm ci
bash scripts/verify_release.sh
npm run dev -- --host 127.0.0.1
```

No `npm create vite` is required for users cloning the repository. `npm ci` installs from the committed `package-lock.json`; `npm install` may intentionally update the lockfile and is more suitable for changing dependencies.

npm scripts

Script	Purpose
`npm run dev -- --host 127.0.0.1`	Local Vite development server
`npm run lint`	Oxlint analysis
`npm run build`	`tsc -b && vite build` production/type check
`npm test`	`vitest run` unit tests
`npm run preview`	Preview built `dist/` locally
`bash scripts/verify_release.sh`	Node/npm versions, lint, build, tests, namespace shell syntax, Git status

The user's successful build generated a `dist/` directory; **do not commit** `dist/` **or** `node_modules/`. The repository's `.gitignore` should exclude generated output and credentials. If installing from an archived ZIP rather than a clone, Git history/remotes will not be available until separately initialized; official installation uses Git clone.

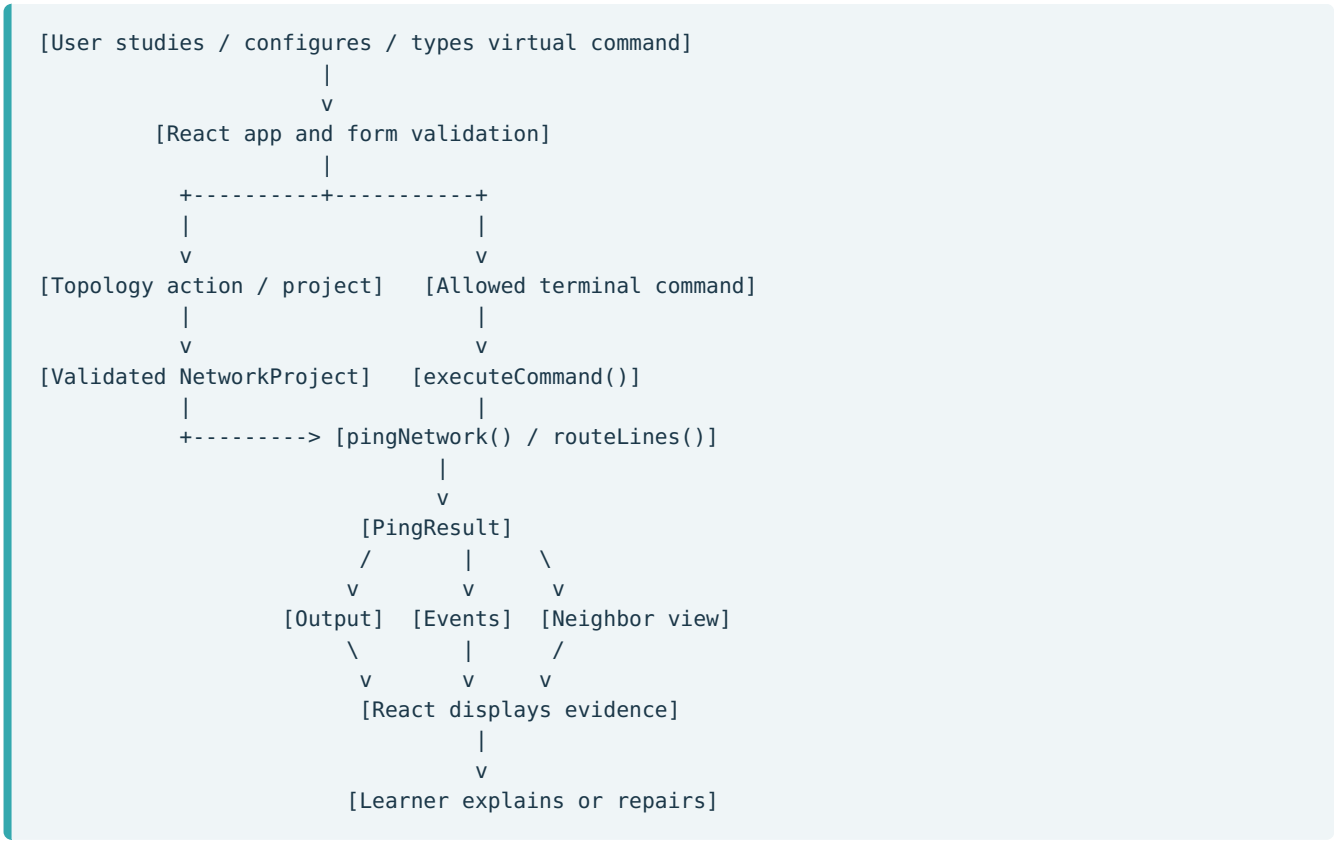
Environment reproducibility: The owner's verified versions (listed above) are the reference. Compatibility outside the reported versions must be separately tested. A lockfile improves dependency reproducibility but does not replace Node compatibility checking.

3. Source architecture and module ownership

```
netverse-lab/
├── README.md
├── docs/
│   ├── NETVERSE_MASTER_BLUEPRINT.md
│   ├── PHASE_0_SETUP.md
│   ├── ENGINE_FIDELITY.md
│   ├── PHASES_1-8_IMPLEMENTATION.md
│   ├── PHASES_9_14_ACCEPTANCE.md
│   ├── USER_MANUAL.md                <- this package
│   ├── DEVELOPER_MANUAL.md           <- this package
│   └── assets/screenshots/          <- illustrated reference
├── src/
│   ├── App.tsx                       # application shell, forms, incident and terminal
integration
│   ├── App.css, index.css
│   ├── main.tsx
│   ├── engine/
│   │   ├── model.ts                  # NetworkProject schema, devices, links, validation
│   │   ├── ip.ts                     # IPv4 parsing/subnet utility
│   │   ├── actions.ts                # validated config/state transformations
│   │   ├── simulator.ts              # headless routing, ARP/ICMP, event traces
│   │   └── terminal.ts               # bounded/allow-listed command interpreter
│   ├── learning/content.ts          # three nine-step lessons
│   ├── labs/scenarios.ts            # three incidents + score rubric
│   ├── features/
│   │   ├── Labs.tsx                  # performance/infrastructure/security/advanced UI
│   │   ├── PatchPanel.tsx            # Patch and Guided/Independent controls
│   │   ├── patch.ts                  # contextual rules-based assistance
│   │   ├── performance.ts            # max-min allocation
│   │   ├── infrastructure.ts         # component metadata
│   │   ├── security.ts               # first-match firewall evaluator
│   │   ├── advanced.ts               # VLAN/TCP conceptual checks
│   │   └── ipv6.ts                   # IPv6 prefix parsing/compare
│   └── utils/storage.ts             # localStorage v1, JSON import/export
├── tests/unit/
│   ├── toolchain.test.ts             # Phase 0 sanity check
│   ├── network-engine.test.ts        # 34 engine checks
│   └── extended-labs.test.ts        # 27 additional checks
├── scripts/
│   ├── verify_netverse.sh
│   ├── verify_release.sh
│   └── real_linux_namespace_lab.sh
├── package.json
├── package-lock.json
├── .oxlintrc.json
└── vite.config.ts
```

Dependency direction: The UI calls pure or near-pure engine functions; the **engine never imports React**. Separate concept labs do not silently write to the core topology model. This distinction lets developers test network behavior in a headless environment.

System flowchart



The state-dependent path is the core educational value: **the engine must not report success solely because the learner clicked the expected controls**.

4. Data schema and persistence

TypeScript model (source: `src/engine/model.ts`)

```
type DeviceKind = 'host' | 'switch' | 'router' | 'server';
type Medium = 'ethernet';

interface NetPort {
  id: string; name: string; mac: string; medium: Medium;
  enabled: boolean; ipv4: string; prefix: number;
}
interface NetDevice {
  id: string; name: string; kind: DeviceKind;
  x: number; y: number; powered: boolean;
  ports: NetPort[]; gateway: string;
}
interface NetLink {
  id: string; a: string; b: string;
  medium: Medium; enabled: boolean; delayMs: number;
}
interface NetworkProject {
  schemaVersion: 1;
  name: string;
  devices: NetDevice[];
  links: NetLink[];
}
```

`PingResult` contains `sourceId`, `destination`, sent/received counts, printable `output`, ordered `events`, hop descriptions, neighbor view, `success`, and failure `reason`. `NetworkEvent` contains event ID, synthetic `timeMs`, event type, location, packet ID, detail, and optional drop reason.

Model constraints: Up to 12 devices, 24 cables, one direct cable per physical port, and Ethernet-only interfaces; router defaults to 2 ports, switch to 4, host/server to 1. `validateProject(unknown)` rejects wrong schema version, invalid IDs, bad IPv4 literals/prefixes, missing link endpoints, duplicate/overused ports, invalid delays, and oversized topologies. JSON imports are capped at 250 KB at the UI layer.

Persistence model

Browser key	Value
`netverse.v1.project`	Active topology (`schemaVersion:1`)
`netverse.v1.lessonProgress`	Lesson ID → step/completion state
`netverse.lab.mode`	`guided` or `independent`

`loadProject()` validates saved topology and uses `createTwoPcLab()` as fallback. `saveProject`/`saveProgress` are invoked by React effects on state updates. User JSON export/import serializes only `NetworkProject` topology—not lesson progress, hints, terminal history or incident notes.

Migration warning: If changing the JSON schema, add a versioned migration; do not silently reinterpret old user backups. `validateProject` is critical to import safety.

5. Core packet processing

Entry points and responsibilities

- `makeDevice`, `createTwoPcLab`, `emptyProject`, `addDevice`, `connect`, `validateProject` in `model.ts`.
- `setAddress`, `setGateway`, `toggleInterface`, `toggleLink`, `removeLink`, `removeDevice` in `actions.ts` (functions return immutable `{ok,state,error?}` results).
- `pingNetwork(project, sourceId, targetIpv4, count)` and `routeLines()` in `simulator.ts`.
- `executeCommand(project, sourceId, input, lastPing)` in `terminal.ts`.

Simplified ICMP pipeline

```
START: selected source + destination IPv4
  |
  +--> source exists and can send?
  |        no --> failure / event
  v
Destination on connected source prefix?
  | yes                          | no
  v                              v
Choose local destination     Look up source default gateway
                                 |
                                 +--> absent or not on-link: NO_ROUTE
                                 v
                            Resolve next-hop MAC / path
  |                              |
  +------------------------------+
  v
Traverse enabled links / interfaces / switches
  |
  +--> missing/inactive connection: LINK_DOWN or ARP_UNRESOLVED
  v
Destination or next router receives packet
  |
  +--> router forwards to connected destination prefix (limited model)
  v
Check reverse/return path
  |
  +--> missing return route: failure
  v
Record ARP/ICMP trace and count Echo Reply
```

The implementation models direct communication, one default gateway, limited router transit, ARP resolution on cache miss and state-dependent ICMP results. The `traceroute` output is derived from a simulated ping path and **does not send TTL-based traceroute probes**. Neighbors reported through `ip neigh` come from the most recent trace, not a persistent kernel ARP cache. Event timing is deterministic and bounded (maximum 200 stored events per ping simulation).

Engineering invariants

1. Disconnected/unpowered/disabled interfaces must not deliver frames.
2. A host with no route to a remote IPv4 network cannot ping that network.

3. A successful Echo Reply requires a valid forward **and reverse** path.
4. ARP resolution must not succeed without the required link/path.
5. Duplicate link endpoints or reused physical ports must be rejected.
6. Unsupported commands must not execute arbitrary host instructions.
7. Every result should be reproducible for an unchanged topology and command.
8. Project import cannot introduce non-model devices or links to missing ports.

Pseudocode: virtual ping

```
function virtualPing(project, source, destination, count):
    validate source and destination input
    choose route: connected prefix or configured default gateway
    resolve next hop using modeled Ethernet/ARP
    follow enabled links and forwarding devices
    if forward path invalid: emit evidence; return failure
    check destination and return path
    if return path invalid: emit evidence; return failure
    append request/reply events for each probe (within event limit)
    return PingResult with counts and trace
```

This summarizes intent, not line-by-line implementation. See `simulator.ts` and `tests/unit/network-engine.test.ts` for the authoritative behavior.

6. UI and state management

`App.tsx` currently owns most application-level React state: current section, active topology, selected device, virtual terminal history, last ping/packet trace, lesson progress, active incident, investigation notebook, Guided/Independent mode, Patch visibility and user notifications.

Main workflow:

1. A sidebar button changes the current section.
2. Build actions return a new validated `NetworkProject` or an error.
3. The app updates React state and invalidates stale verification/packet results.
4. The terminal's selected source invokes `executeCommand` against current project state.
5. A ping stores `PingResult` and enables Packet Inspector replay.
6. Saved project and progress are written to `localStorage` on state updates.

The topology is an SVG with a fixed coordinate viewBox; device inspector X/Y controls work, but dragging isn't implemented. This is a known ergonomics constraint. `App.tsx` was refactored to remove four `react(set-state-in-effect)` warnings by using keyed editor remounts/derived playback state; avoid reintroducing effect loops. The Phase 1–8 TS1294 error was corrected by replacing a constructor parameter property with an explicit declared field under `erasableSyntaxOnly`.

Current navigation limitation: Sidebar sections are React state; the browser URL may remain at a previous path while another section is displayed. A future route-aware redesign should synchronize location, back/forward and deep links.

7. Learning, assistance and case assessment

Lesson model

`src/learning/content.ts` owns the nine labels and three implemented introductory lessons with quiz content. Progress is keyed by lesson ID. The knowledge check unlocks at step 9 (index 8); a correct response marks the lesson complete. Content for the other 13 modules is not yet populated.

Patch and learning modes

`PatchPanel.tsx` renders the help panel and mode switch; `getPatchHelp` in `patch.ts` derives suggestions from current section, topology links, last ping, mode and hint level. Patch is offline, rules-based and read-only. In Independent mode, detailed incident help is deliberately opt-in. Do not couple Patch to network mutations without explicit new design and testing.

Incident model and grading

`src/labs/scenarios.ts` exposes three IDs: `cable`, `subnet`, `gateway`. Each `loadIncident(id)` creates an actual flawed `NetworkProject`, not just a message. The app records whether the learner ran a failed test and later verified recovery. Grading is:

- 40: verified successful repair;
- 20: observed failing baseline test;
- 20: evidence text contains at least 15 characters;
- 20: diagnosis text contains at least 15 characters.

`gradeIncident()` **does not semantically assess the explanation text**; this is a significant instructional limitation. Add content validation or instructor review before describing grading as expert-level diagnostic assessment.

8. Phases 9–14 feature boundaries

Feature	Source	Algorithm / contract	Not implemented
Performance	`features/performance.ts`	Deterministic max-min fair sharing, one bottleneck, approximate RTT under overload	Real network measurement, packet scheduler, TCP CC, Wi-Fi airtime
Infrastructure	`features/infrastructure.ts` + `Labs.tsx`	Educational equipment descriptions and diagram	Optical budget, RF propagation, accurate 3D models
Security	`features/security.ts`	First-match enabled rule, source IPv4 CIDR, TCP/UDP, destination port and default action	OS firewall, Build-engine policy integration, real packets

Feature	Source	Algorithm / contract	Not implemented
Advanced VLAN	`features/advanced.ts`	Same broadcast-domain membership if VLAN IDs and links are valid	Trunks, 802.1Q tags, forwarding and STP
Advanced TCP	`features/advanced.ts`	SYN/SYN-ACK/ACK educational state	Real sockets, congestion, retransmission
Advanced IPv6	`features/ipv6.ts`	Hexadecimal prefix comparison (`/0`–`/128`)	IPv6/NDP packet stack, embedded IPv4 literal support
Linux exercise	`scripts/real_linux_namespace_lab.sh`	Explicit opt-in, temporary veth + namespaces, cleanup via trap	Browser control, production network setup
QA	`scripts/verify_release.sh`	lint → build → tests → bash syntax → git status	Full browser E2E/Android audit/ penetration test

These separate models are intentionally **not wired into the central ARP/ICMP simulator**, so claiming full combined real-world fidelity would be inaccurate.

9. Quality assurance and acceptance

Commands

```
cd ~/Documents/netverse-lab
npm run lint
npm run build
npm test
bash -n scripts/real_linux_namespace_lab.sh
bash scripts/verify_release.sh
```

The owner's captured Ubuntu results showed:

```
Oxlint: Found 0 warnings and 0 errors
Vite: built successfully (33 modules transformed)
Vitest: 3 files passed, 62 tests passed
```

`62` = 1 toolchain sanity test + 34 core network tests + 27 extended-lab tests. **These are the user's reported actual Ubuntu results**, not a claim that the author re-executed the user's local application.

Mandatory manual regression matrix

ID	Test	Expected
QA-01	Guided → Build	Coaching strip visible
QA-02	Independent → Build	Coaching strip hidden, Patch still available
QA-03	Refresh after mode selection	Mode persisted

ID	Test	Expected
QA-04	Two-PC LAN ping with cable	2/2 returned
QA-05	Disable cable and ping	0/2 returned
QA-06	Re-enable and re-ping	2/2 returned
QA-07	Inspect replay	New ping produces new ARP/ICMP events
QA-08	Cable incident	Failure → fix → verified success
QA-09	Wrong-subnet incident	Failure → corrected address → success
QA-10	Missing-gateway incident	Failure → correct default gateway → success
QA-11	Capacity 20, demands 20/20	10/10 Mbps
QA-12	Capacity 20, demands 2/20	2/18 Mbps
QA-13	Security office TCP/443	Allow
QA-14	Security guest TCP/22	Deny
QA-15	Different VLAN IDs	No same-L2 membership
QA-16	Missing SYN-ACK	TCP teaching handshake incomplete
QA-17	Same IPv6 `/64`	Prefix comparator true
QA-18	Export then re-import JSON	Topology restored
QA-19	Resize browser to 375px	Controls accessible; note layout issues
QA-20	Inspect optional Linux script with no `--run`	Prints instructions, makes no changes

Screenshots supplied by the owner show a functioning UI, but some nodes overlap and a stale `/lesson/py-01` URL may remain while different sidebar sections display. Treat these as UX backlog items, not proof of network-engine failure.

Future tests: add Playwright browser E2E for navigation/back-forward, responsive mobile states, Patch hints, importer security tests, regression screenshots and keyboard focus. Use randomized property checks in CI for IP/subnet/route invariants.

10. Debugging and failure investigation

TypeScript `TS1294` with `erasableSyntaxOnly`

Cause: TypeScript parameter properties require syntax transformation disallowed under erasable-only rules.

Bad:

```
constructor(private project: NetworkProject) {}
```

Correct pattern:

```
private project: NetworkProject;
constructor(project: NetworkProject) {
  this.project = project;
}
```

Oxlint `react(set-state-in-effect)`

Cause: Synchronous `setState()` in an effect merely to copy props into editable state or reset playback. Prefer keyed remounts, derived render values or event-driven state changes; do not globally disable the rule to hide logic problems.

Simulated ping fails despite visible cable

Inspect in order: device power, port admin status, link enabled, source IPv4/prefix, target IPv4/prefix, next-hop route, router connected interfaces, and destination return path. The canvas line is a visual cue; the engine's `PingResult.reason` and event timeline are authoritative for simulated failure.

Empty inspector

Check if a `ping` or simplified `traceroute` command has actually been executed. A changed topology clears the previous trace to avoid showing stale evidence.

Import problems

Use the schema validator as a guard, not a JSON parser that constructs new JavaScript expressions. The UI rejects invalid / oversized files. Add tests before relaxing limits.

Browser data disappears

Confirm the browser profile and site origin; `127.0.0.1` and `localhost` are distinct origins for browser storage. JSON export only saves topology; lesson progress remains browser-local.

Git repository remote unknown

Use `git remote -v` before attempting to push. `git status` showing `main` or a feature branch **does not prove** a GitHub remote exists. Do not invent a remote or force-push.

11. Release engineering, GitHub and final commit

A. Current state (from owner's logs)

```
main:                  Phase 0 design commit baf7dc2
feature/netverse-phases1-8: engineering commit e77c5fd
fix/netverse-phase1-8: accepted hotfix commit 1212d6c
feature/netverse-extended-labs: checked, uncommitted release files
```

Actual branch tips can evolve; run `git log --oneline --decorate -8` to verify history before publishing.

B. Install documentation overlay, review, then run gate

```
cd ~/Documents/netverse-lab
git branch --show-current
git status --short
# Extract the supplied docs ZIP into repository root (see delivery instructions).
git diff --check
bash scripts/verify_release.sh
```

Expected: no lint/build/test failures. A clean `git diff --check` means no whitespace errors. Inspect `git diff --stat` and `git status`; documentation assets should be intentional.

C. Commit on feature branch

```
git add README.md docs src tests scripts
git diff --cached --check
git diff --cached --stat
git commit -m "feat: finalize Netverse extended labs and manuals"
git status
```

The commit should include the already-tested application changes **plus** `docs/USER_MANUAL.md`, `docs/DEVELOPER_MANUAL.md`, and screenshot assets. Avoid committing private credentials, logs, `node_modules` or `dist`.

D. Merge to main *after* successful verification

```
git switch main
git merge --no-ff feature/netverse-extended-labs -m "merge: Netverse extended educational release"
bash scripts/verify_release.sh
git status
```

If merge conflicts occur, stop, inspect them and re-run checks. Do not use `git reset --hard` to resolve conflicts blindly.

E. Create the GitHub repository and push

1. Create an **empty** GitHub repository in the browser (suggested name `netverse-lab`) under your account; leave "Initialize with README" unchecked so the remote has no unrelated initial commit.
2. Copy its actual HTTPS URL.
3. Run `git remote -v`. If no `origin` exists, register the copied URL:

```
git remote add origin <ACTUAL_GITHUB_CLONE_URL>
```

Replace the whole placeholder (including angle brackets) with the URL; do not literally paste `<...>`. If `origin` already exists, check that it points to **your intended repository** before pushing. Do not overwrite an unfamiliar remote.

Then:

```
git push -u origin main
```

Optionally publish the feature branch for review:

```
git push -u origin feature/netverse-extended-labs
```

A version tag is useful but optional. For an accepted **educational prototype**, use an explicit prerelease tag rather than implying production completeness:

```
git tag -a v0.1.0-rc.1 -m "Netverse educational extended release candidate"
git push origin v0.1.0-rc.1
```

Important: Tag the intended merge commit on `main` only after QA. Publishing to GitHub, merging and tagging are user actions; this document does not assert any of them already occurred.

F. Test clean-clone install

After pushing, use a *separate* temporary folder so the existing repo is not overwritten:

```
mkdir -p ~/Documents/netverse-install-check
cd ~/Documents/netverse-install-check
git clone <ACTUAL_GITHUB_CLONE_URL> netverse-lab
cd netverse-lab
npm ci
npm run lint && npm run build && npm test
npm run dev -- --host 127.0.0.1
```

If the clean clone works, your repository includes what a new learner needs. To prevent confusion, make the URL in README match the actual GitHub repository.

12. Security, performance and accessibility

- **Security boundary:** Virtual commands are selected via a hardcoded allowlist and never execute shell commands. Browser import is schema-validated and limited in size. Untrusted user data must not be evaluated as code. No backend is required for the current site.
- **Real Linux boundary:** Namespaces are opt-in through a separate shell script; it requests root and creates temporary kernel resources. Its EXIT trap cleans up on normal and typical abnormal termination. Do not integrate arbitrary sudo execution into the browser.
- **Performance:** The headless engine is intentionally capped in devices/links/events. Avoid unbounded graph traversal, event loops or per-render simulation runs. Browser state for 3D, large topologies or additional protocols requires memory/performance profiling on the target 8 GB laptop.

- **Accessibility:** SVG nodes have keyboard selection and a separate accessible device list. Continue improving focus styles, responsive toolbar wrapping, readable node names, labels, and keyboard operation of new controls.
- **Observability:** For a bug report capture `git rev-parse --short HEAD`, `node -v`, `npm -v`, `npm run build` output, screenshot, steps to reproduce, browser, and exported *non-sensitive* project JSON.

13. Known issues and extension roadmap

Prioritized backlog:

1. **UX:** Fix nodes overlapping and label truncation on the topology SVG; implement optional drag-to-reposition with keyboard equivalent.
2. **Routing:** Sync sidebar sections with URL and browser back/forward; screenshots currently show a stale `/lesson/py-01` pathname.
3. **Learning:** Fill the remaining curriculum modules using the nine-step method, with meaningful quizzes/mini-projects; do not show placeholders as complete.
4. **Assessment:** Validate diagnostic reasoning beyond character count, or create an instructor review flow.
5. **Protocol fidelity:** Add real simulated DHCP/DNS/NAT, richer switch forwarding, route tables and independent return-path tests.
6. **Integration:** Connect VLAN and security policy models to forwarding engine only after explicit design/test review.
7. **Advanced:** Implement IPv6 packet forwarding/NDP, TCP state/retransmissions and realistic queueing only when teaching objectives require them.
8. **Infrastructure:** Extend schematics into selected 3D/physical failure scenarios with explicit fidelity notes.
9. **Quality:** Add Playwright E2E, manual Android verification, CI pipeline, accessibility audit and systematic data-migration tests.
10. **Distribution:** Decide between local dev workflow, static GitHub Pages deployment, or packaged offline application. For GitHub Pages, use the correct Vite `base` path and verify browser storage origin changes.

14. Rebuild checklist

A new developer should be able to reconstruct or extend NETVERSE from this document **together with the Git repository**, not from this document alone; the full implementation is source of truth.

1. Clone `main` and install pinned dependencies via `npm ci`.
2. Read `NETVERSE_MASTER_BLUEPRINT.md`, `ENGINE_FIDELITY.md`, and this manual.
3. Run lint/build/test before modifying anything.
4. Understand the `NetworkProject` schema and `validateProject` before editing device types.

5. Trace the control path `App.tsx` → `actions.ts` / `terminal.ts` → `simulator.ts` → `PingResult` → Packet Inspector.
6. Reproduce all three incident cases and check return-path logic.
7. Keep new physical/security/advanced models honest about their fidelity and whether they integrate into core packet delivery.
8. Add tests **first** for new protocol behavior, including failure, adversarial input and property invariants.
9. Update both manuals and acceptance checklist whenever UI/commands change.
10. Build, lint, test, manually inspect responsive layout, then commit on a feature branch and merge after review.

Engineering rule: A visually successful animation is not evidence of valid network delivery. A scenario is accepted only when the same underlying state drives configuration, terminal output, packet events and test results.